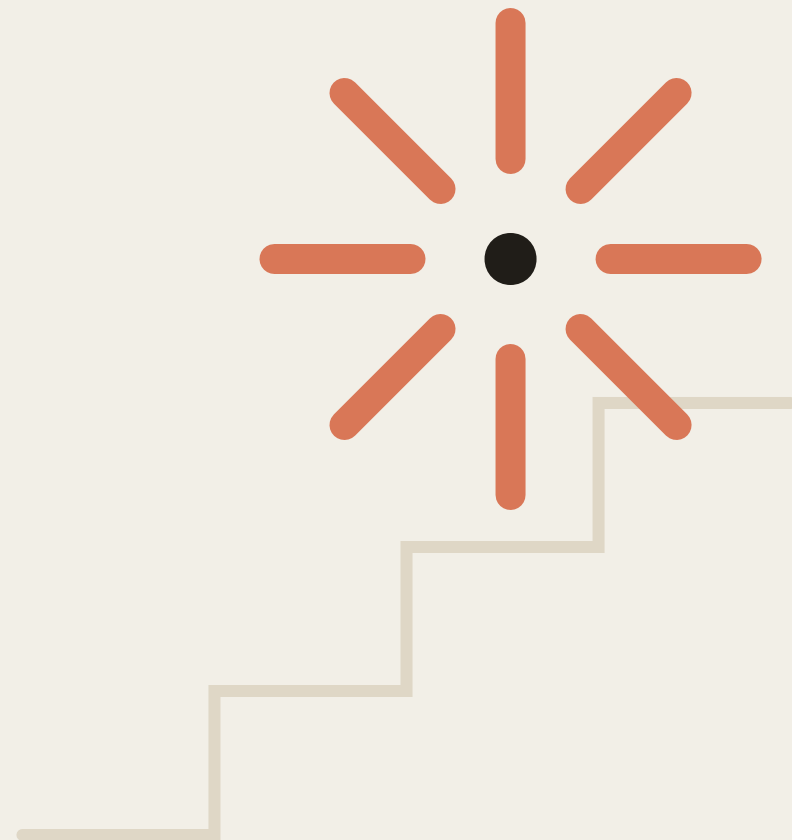


Now the other pile.

Two more skills, seven more rules, same spec, same prompt. The five defects that survived three checkpoints are about to become impossible to ship quietly.

> Read SPEC.md and build it.



One variable at a time, on purpose.

CHECKPOINT 1 · SPEC ONLY

BOUNDARY: ~5
QUALITY: n/a
SPEC: ~3
TOTAL: ~10

It guessed the architecture, because we never told it.

CHECKPOINT 2 · + API-BOUNDARY

BOUNDARY: 0
QUALITY: n/a
SPEC: ~3
TOTAL: ~3

One class of finding gone. The rest untouched.

CHECKPOINT 3 · TWO APPS READ

the same five defects
in both, and two of
them **invisible**
to today's rulebook

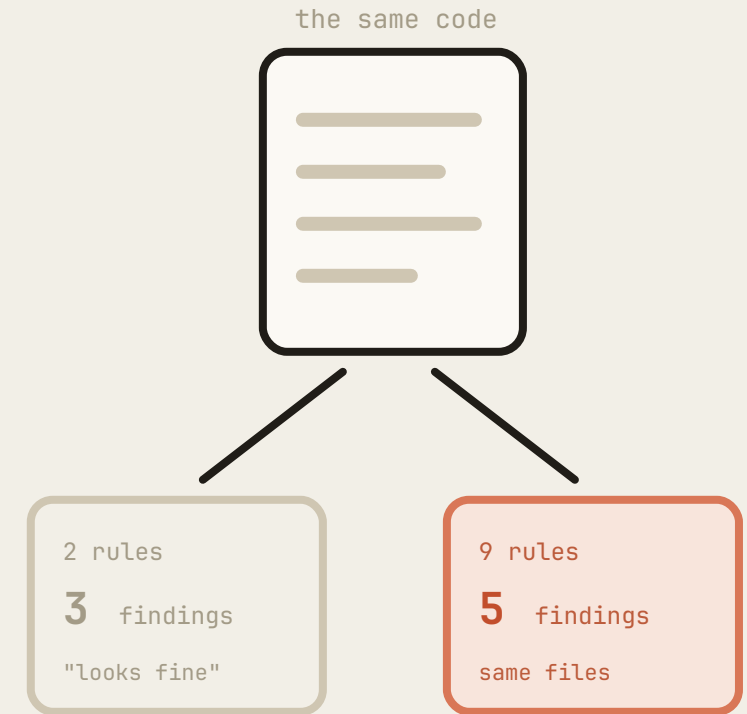
An audit only finds what a rule names.

Numbers are the shape to expect, not a guarantee, because a live build is not deterministic. What repeats is which bucket moves and which does not.

Today closes the last gap, and it closes it **without touching SPEC.md or the prompt**.

They were always there. Now they have a **name**.

At checkpoint 3 the suppressed warning and the magic string went unreported, because no rule on that branch named them. Point the same reviewer at the same unchanged files with these two skills on the branch, and both become findings. The code did not improve or decay. The rulebook grew.



the lens changed, not the code

- "the review came back clean" • with two rules loaded
- + "the review came back clean" • with nine rules loaded

Three things that must never ship.

```
// NEVER: discards the message the UI must show
catch (AssetDeskException) { _error = ""; }

// NEVER: invents copy the spec says not to write
catch (AssetDeskException) { _error = "Something went wrong."; }

// CORRECT
catch (AssetDeskException ex) { _error = ex.Message; }

// NEVER: ! is a suppression too
Employees.FirstOrDefault(e => e.Id == a.AssignedTo)!.Name

// CORRECT
Employees.FirstOrDefault(e => e.Id == a.AssignedTo)?.Name ?? ""
```

1 · NO SWALLOWED EXCEPTIONS

An empty or message-discarding catch turns a failure into silence. Catch the specific type; if you cannot handle it, let it propagate.

2 · NO SUPPRESSIONS

No `!`, no `#pragma warning disable`, no `<NoWarn>`. "A warning is information; suppressing it deletes the information and keeps the bug."

3 · VALIDATE AT THE BOUNDARY

Every endpoint validates its own body. UI validation is a convenience for the user, **never a defence**. `curl` bypasses it, and §9 tests exactly that path.

THE SHAPE OF EACH RULE

Every rule shows **the wrong code and the right code**, side by side. That is why it works on a model and on a new hire: no adjectives, just examples.

Components stay small and dumb.

1 · ONE COMPONENT, ONE JOB

Fetching, holding page state, filtering, and rendering three views is four responsibilities in one file. §3.2 already **names** the components this app has, so collapsing two of them is a violation even if the result is short.

Tripwire at 250 lines. The count is a signal; the responsibility count is the rule.

2 · NO BUSINESS LOGIC IN MARKUP

`@if` on a field is fine. Filtering, sorting, totalling and formatting go behind a named member in `@code`, or into `Data/Format.cs` where the spec put them.

3 · DECLARE EACH DTO ONCE

One definition, in `Data/Dtos.cs`. A second record `Row(...)` inside a component "will drift from the real shape, and the drift shows up as a rendering bug nobody can trace."

4 · ENUM VALUES COME FROM THE ENUM

Never `a.Status.ToString() == "InStock"`. A string literal is unchecked: **rename a member and the compiler stays silent while the count silently goes to zero.**

ONE OF THESE FOUR WILL FIND NOTHING

Rule 2 catches none of the planted defects. A rule that does not fire is still the guardrail for the code you write next month.

Note where both skills keep pointing: back at `SPEC.md` §3.2, §4.1, §4.3, §4.6, §9. They enforce the spec's own promises rather than replacing them.

Five defects, seven rules, no gaps.

THE DEFECT	CAUGHT BY	AT CHECKPOINT 3 IT WAS
Swallowed exception, blank banner	csharp-quality · 1	already a SPEC finding
<NoWarn> hiding a null dereference	csharp-quality · 2	invisible
Repository guard commented out	csharp-quality · 3	already a SPEC finding
Table inlined into Home.razor, duplicate row type	hygiene · 1 and 3	already a SPEC finding
Status compared as a string literal	hygiene · 4	invisible

Two rows changed from **invisible** to **owned**, and one defect is now caught twice over. That redundancy is fine: overlapping rules are cheaper than a gap.

"No suppressions" has **exactly one** carve-out.

```
AssetDesk.csproj

<!-- SQLitePCLRaw.lib.e_sqlite3 is pinned transitively
      by Microsoft.Data.Sqlite; no newer patch exists
      yet to clear this audit advisory. -->
<NoWarn>$(NoWarn);NU1903</NoWarn>

# and the one that is a violation:
<NoWarn>$(NoWarn);NU1903;CS8602</NoWarn>
```

Both lines suppress a warning. One is engineering; the other hides a `NullReferenceException`. The comment is the difference.

THE CARVE-OUT HAS A PRICE

The skill allows NU1903, and requires the .csproj to carry a comment naming **which package and why**. The exception is allowed; the silence is not.

WHY ONE CARVE-OUT BEATS NONE

An absolute rule with no escape hatch gets disabled the first time it is inconvenient. A rule with **one documented exception** survives, because following it is possible.

WHEN YOU WRITE YOUR OWN

If a rule has never been argued with, it is probably not being applied. Real standards accumulate carve-outs, each with a reason attached.

Near zero. Which is **not the same as good.**

CHECKPOINT 4 • PREDICTED

BOUNDARY: 0

QUALITY: 0

SPEC: 0

TOTAL: 0 to 2

Say it before you run it. A prediction that cannot be wrong proves nothing.

WHAT A CLEAN REPORT ACTUALLY MEANS

"No rule I was given was violated." Nothing more. The reviewer has no taste, no product sense, and no opinion about whether the feature was worth building.

WHAT NINE RULES STILL CANNOT SEE

Whether the app is any good. Whether the dashboard answers the question an IT admin actually asks. Whether the seed data represents reality. Whether the whole approach was right.

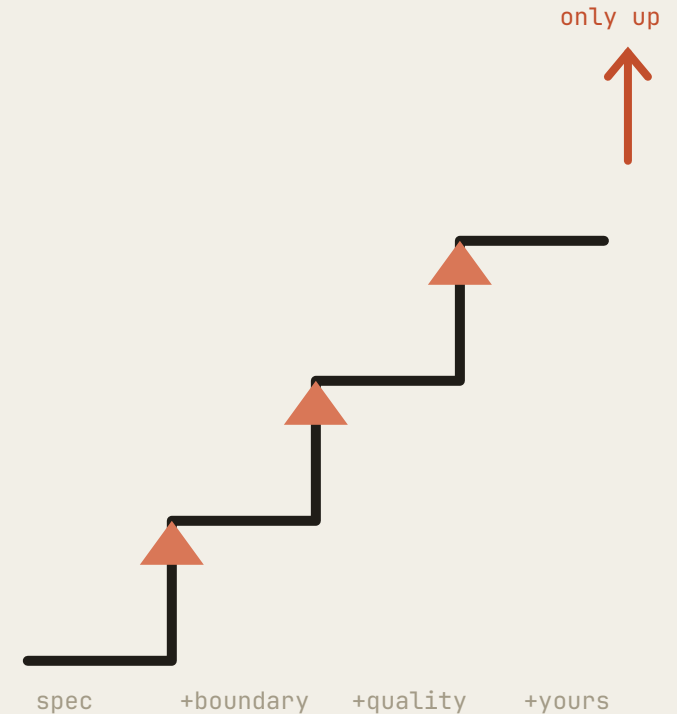
Not one of those is in a skill, and none of them should be. That judgment stays yours. The audit just stops it being spent on missing null checks.

IF SOMETHING STILL COMES BACK

Good. Read it, decide whether the rule or the code is wrong, and change whichever one it is. A skill is a draft, and the findings are how you edit it.

Each rule is a class of bug that **cannot come back.**

You did not fix five defects today. You wrote down seven rules, and the defects stopped being reachable in this build, in the next one, and in the one a colleague runs in March without attending this session. The same reviewer runs headless in CI, against the same skills, on every pull request.



- catch the swallowed exception in review, again, on every project
- + write it once; every future build starts already knowing

Start with the comment you are tired of writing.

1 • ONE SKILL, FROM YOUR REVIEW HISTORY

Find the note you have left on pull requests three times this month. That is your first skill. Name the trigger in the description, then show **wrong code and right code**, with no adjectives.

2 • A REVIEWER THAT CANNOT WRITE

A subagent with read-only tools, forbidden from reviewing work it produced, printing counts in a fixed shape. Comparable numbers are what turn "it feels better" into evidence.

3 • THEN PUT IT IN CI

Headless, on every pull request, against the same skills. The rule stops depending on whether a human remembered to look on a Friday afternoon.

WHAT NOT TO DO

Do not write nine skills on day one. An unused rule still costs context on every turn, and a rule nobody agreed to gets disabled the first time it is inconvenient.

THE TEST FOR A GOOD RULE

Can a reviewer point at a line and say "this violates it", without debate? If two reasonable people would disagree, it is not a rule yet. It is a preference, and it belongs in a conversation.

AND KEEP THE SPEC HABIT

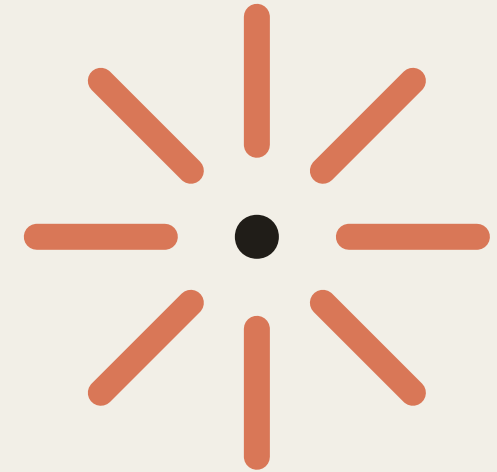
The skills only worked because SPEC.md never moved. Without a fixed target, you cannot attribute an improvement to anything.

Everything you saw today is four markdown files and one subagent. There was no framework.

Same spec. Same prompt. Four **different outcomes.**

Nothing today was about writing code faster. It was about making the standard **a file instead of a memory**, so it survives a long session, a new colleague, and a Friday afternoon. The prompt was eight words every single time.

- + the repo carries its own spec `SPEC.md`
- + the repo carries its own standards `.claude/skills/`
- + the repo audits itself `/review-build`



one change, in your repo,
tomorrow →